Security Assessment

Item 10 Internal Report

August 2026

Prepared for ether.fi

Disclaimer

This document is intended for **internal** use only.

Initial Commit Hash	Final Commit Hash
0b4c8da3f	17ccfc063

Protocol Scope

- src/stock-withdraw/StockWithdrawModule.sol
- src/stock-withdraw/StockUnwrapper.sol
- src/top-up/bridge/StockOFTBridgeAdapter.sol

Informational Issues

I-01. executeWithdrawal can be executed right up to order.deadline, guaranteeing the destination compose takes the expired branch and bypassing minReturn

Description: executeWithdrawal is permissionless and only rejects orders whose deadline has already passed on the source chain. The destination-side branch selection, however, happens minutes later at lzCompose time using the destination chain's timestamp. Any execution in the last seconds before deadline deterministically lands in the expired branch on Ethereum, which skips the ERC-4626 redeem entirely (bypassing the user's signed minReturn) and ships the wrapped shares to sourceSafe. Because executeWithdrawal is permissionless and payable, any third party can force this for any matured order at the cost of the LayerZero fee only, preventing the signed recipient from receiving the underlying stock and forcing the user into the wrapped-token recovery flow.

Recommendation: Consider introducing a minArrival buffer, between the deadline and the lzCompose recovery flow.

Customer's response: Fixed in d33e49580

Fix review: Fix confirmed

I-02. Provider fee and destination route are read live for already-signed orders, letting a config change silently degrade or break in-flight withdrawals

Description: providerFeeBps, feeReceiver, and stockUnwrappers[dstEid] are deliberately not snapshotted with the order and are read at execute time. Raising providerFeeBps after signing reduces the bridged shares. Since minReturn is enforced against the redeem output on Ethereum, an order sized close to its floor could revert at lzCompose and stay parked. Once the deadline passes, it is settled through the fallback branch, meaning a fee change can push a user's position into the stranding path. Furthermore, changing feeReceiver or the unwrapper mapping while orders are pending bridges the user's tokens to the new address. A non-StockUnwrapper recipient would simply keep the tokens.

Recommendation: Consider snapshotting the aforementioned parameters at request time.

Customer's response: Fixed in d33e49580

Fix review: Fix confirmed

I-03. StockOFTBridgeAdapter entry points are unauthenticated and directly callable, letting anyone sweep balances

Description: bridge() has no access control and is intended to run only via delegatecall from TopUpFactory. Nothing prevents calling it directly on the adapter contract. Any raw-stock token or native ETH that reaches the adapter address can be wrapped and OFT-sent to an attacker-chosen recipient.

Recommendation: Consider restricting to delegateCall only by requiring address(this) == FACTORY.

Customer's response: Acknowledged

I-04. executeWithdrawal never re-validates supportedTokens, so de-listing a compromised iTOKEN does not stop in-flight orders

Description: executeWithdrawal performs no check on whether withdrawal.order.iToken is still supported. If an admin removes a compromised iTOKEN via configureTokens, already-stored orders for that token remain fully executable.

Recommendation: Consider re-checking $.supportedTokens.contains(withdrawal.order.iToken) inside executeWithdrawal to ensure de-listing immediately halts in-flight orders.

Customer's response: Fixed in d33e49580

Fix review: Fix confirmed

I-05. Request-time validation never checks for OFT shared-decimal truncation, accepting unexecutable orders

Description: requestWithdrawal accepts any nonzero order.amount without verifying if the amount remaining after the provider fee survives the OFT's shared-decimal truncation. An order whose net bridge amount truncates to zero will forever revert during execution, locking the safe's withdrawal slot.

Recommendation: Consider performing the quoteOFT sizing check at request time, ensuring amountSentLD > 0 before creating the request.

Customer's response: Acknowledged

I-06. Missing roleRegistry != address(0) validation in initializers breaks administrative and rescue paths

Description: The initializers for StockUnwrapper and StockWithdrawModule proxy implementations do not validate that _roleRegistry is non-zero. If deployed with a zero address, the contracts become completely unconfigurable and structurally unable to rescue stranded cross-chain funds.

Recommendation: Consider reverting if _roleRegistry == address(0) in both contract initializers.

Customer's response: Acknowledged

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.